

Data Structures

Lecture 10: Binary Search Tree

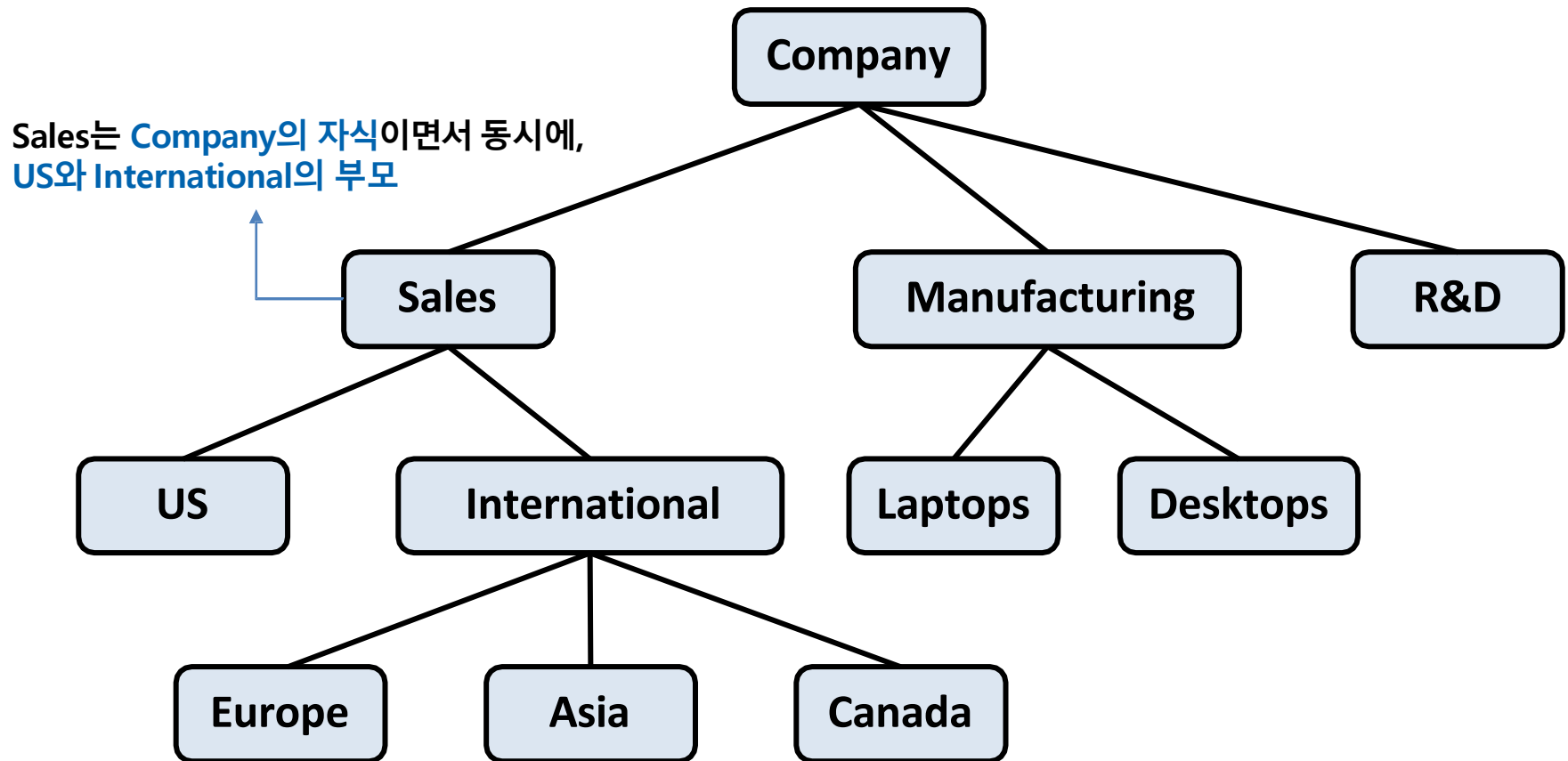
Soobin Um

Apr. 6, 2026

복습

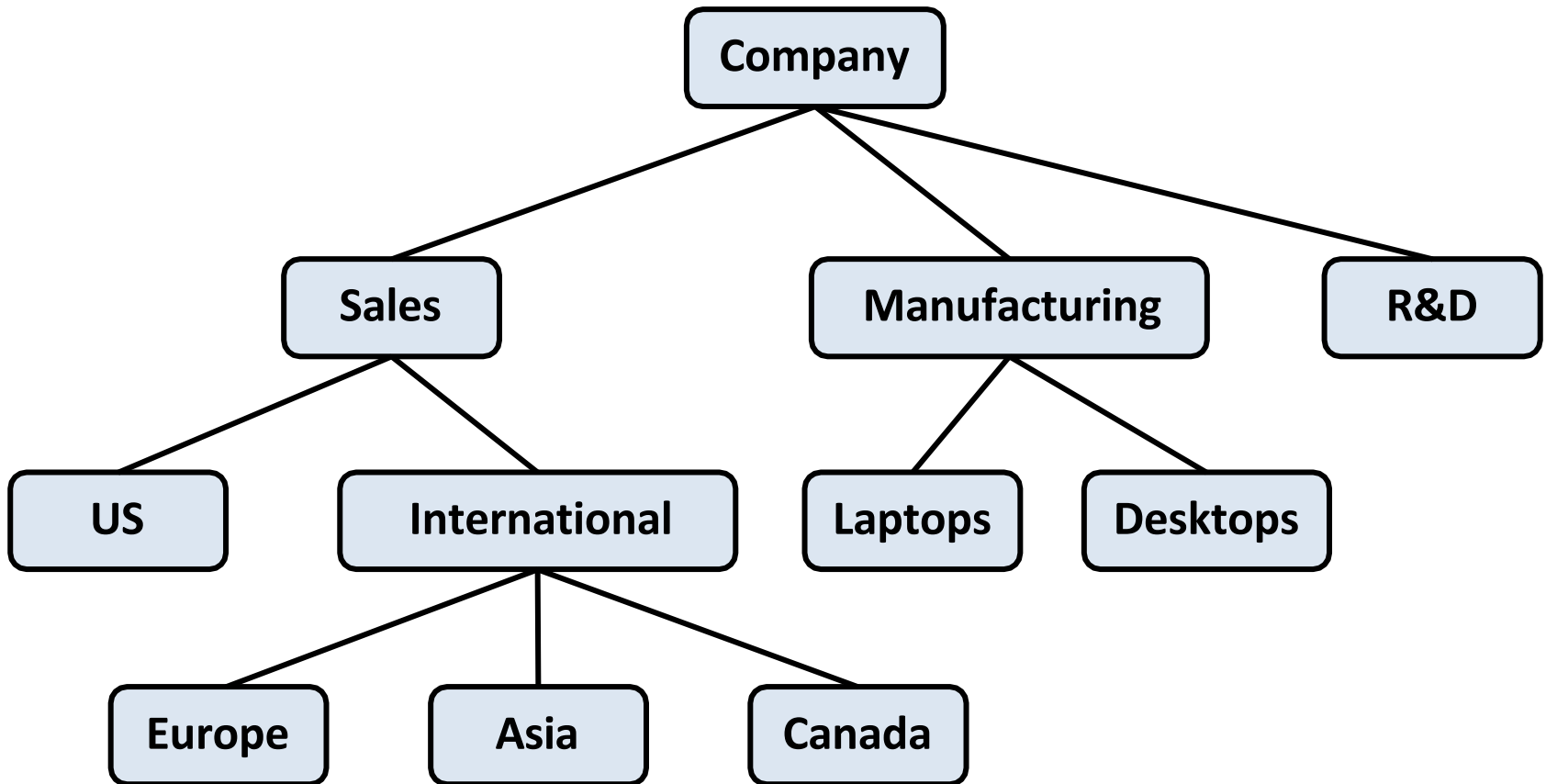
Definition of a Tree

- 데이터를 “계층적 (hierarchical)” 방식으로 저장하는 구조
 - 부모-자식 (Parent-Child) 관계를 통해 연결된 노드들로 구성됨 (예: 조직도, 파일시스템)



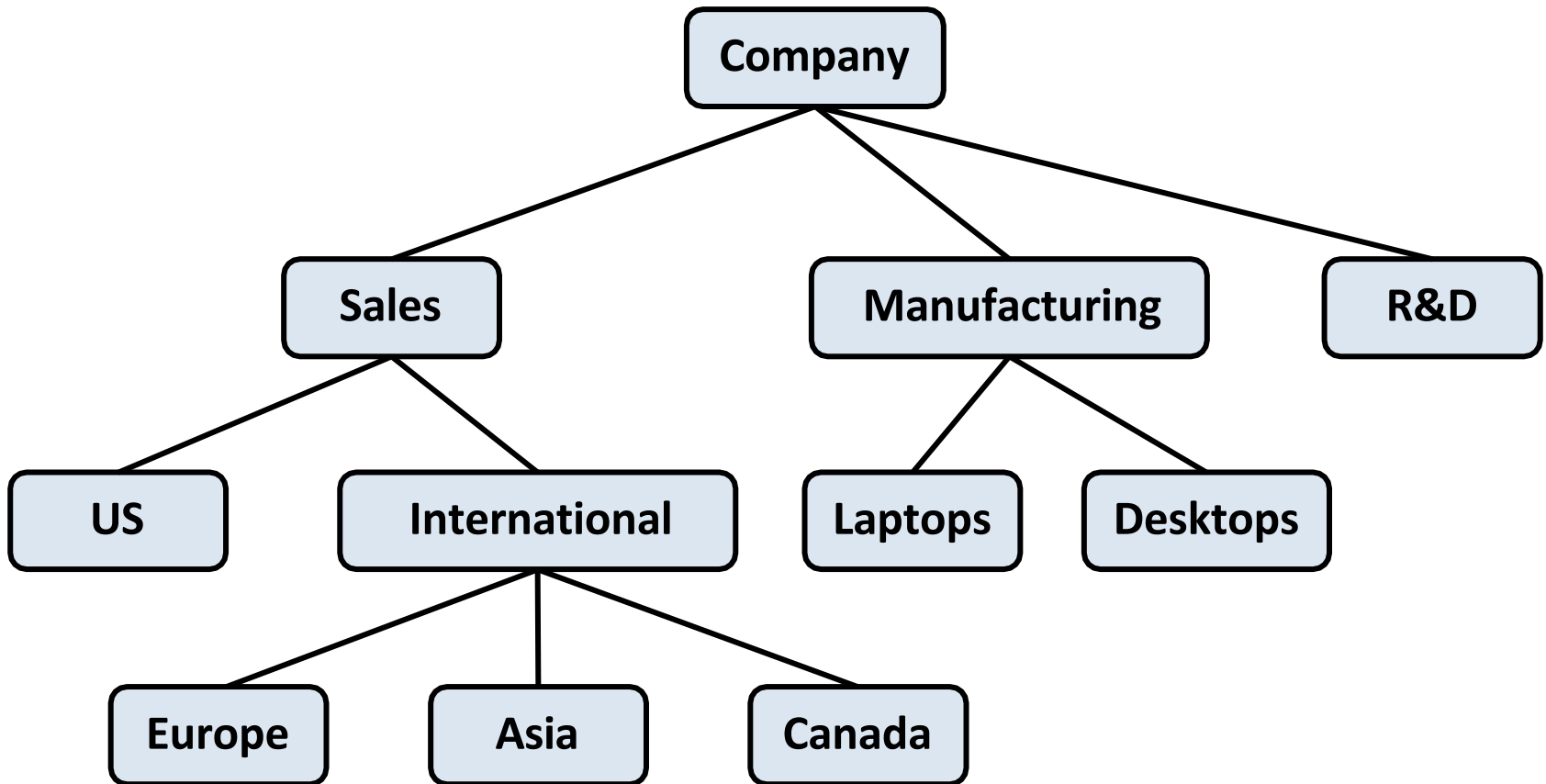
Terminologies

- **노드 (Node):** 데이터가 저장되는 기본 단위 (unit)
 - 노드에는 데이터 값과 다른 노드와의 연결 정보(포인터)가 포함될 수 있음
- **간선 (Edge):** 두 노드 사이의 연결 (relationship)



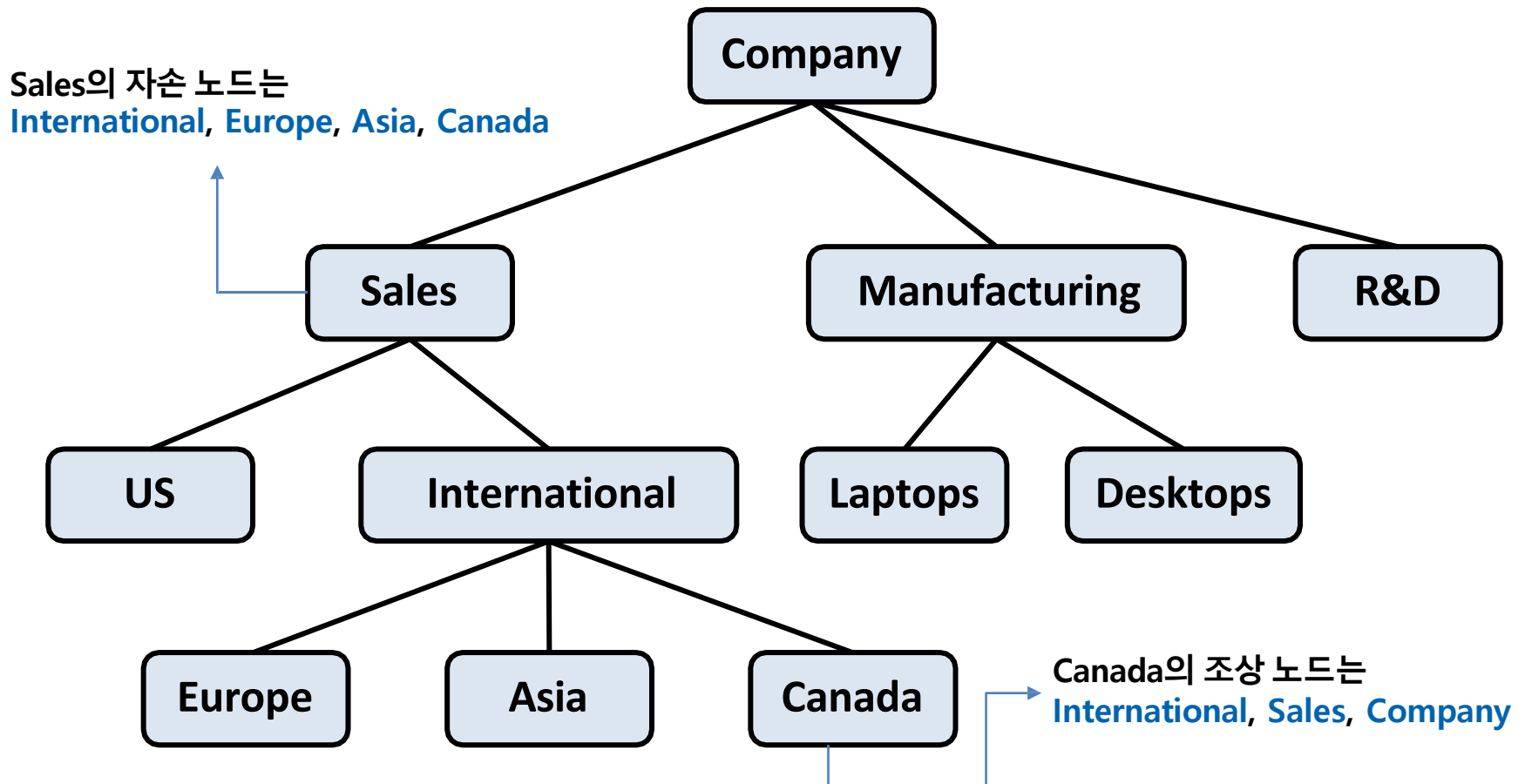
Terminologies

- 루트 (Root): 가장 상위에 있는 (부모 노드가 없는) 노드
- 내부 노드 (Internal node): 하나 이상의 자식 노드를 갖는 노드
- 외부 노드 (External node, leaf node): 자식 노드가 없는 노드



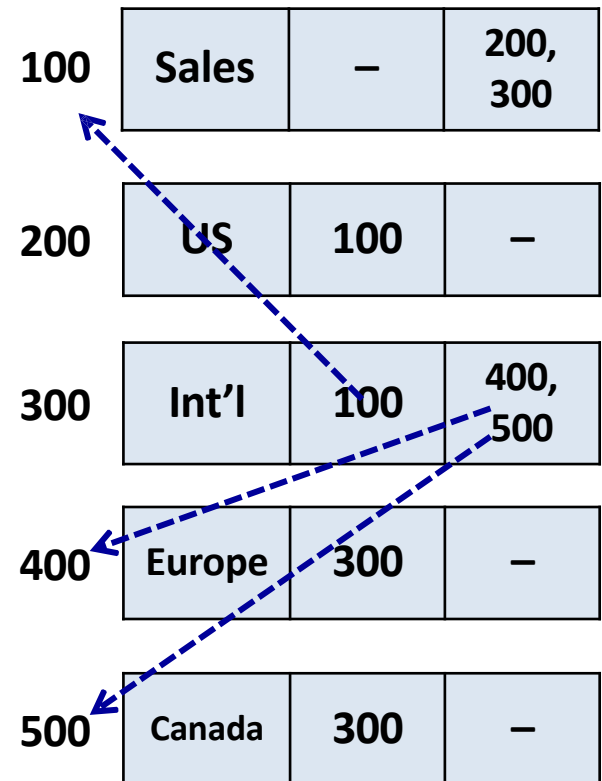
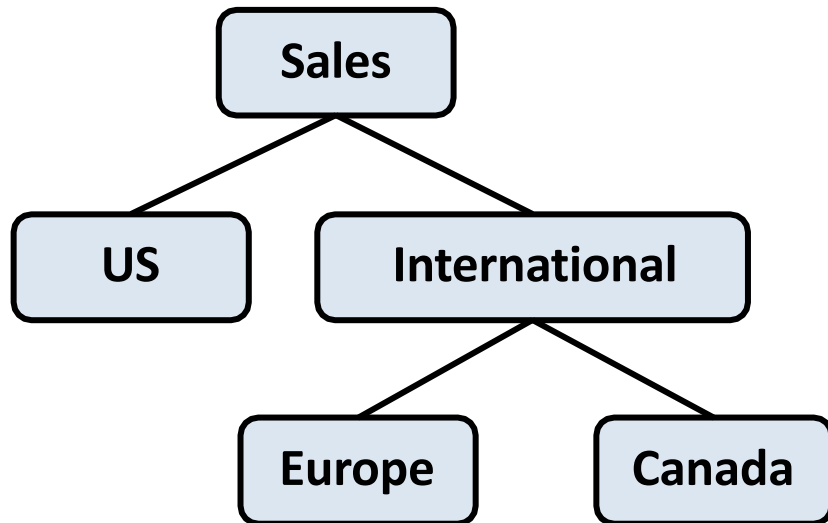
Terminologies

- **조상 노드 (Ancestor)**
 - 어떤 노드에서 루트까지 가는 경로 상에 존재하는 모든 노드 (부모 노드는 조상 노드에 포함)
- **자손 노드 (Descendant)**
 - 어떤 노드로부터 하위 방향으로 이어지는 모든 노드



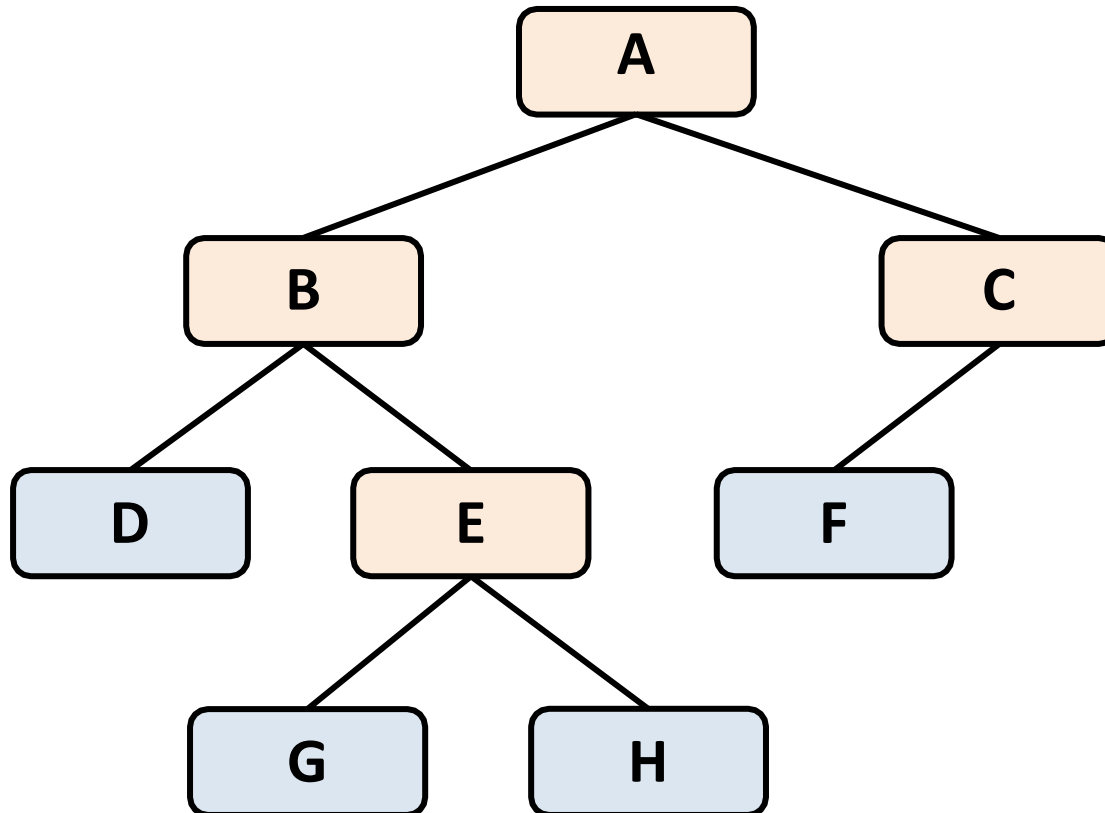
Tree in Memory (Linked Structure)

- 노드는 다음을 저장하는 객체(object)로 표현:
 - 데이터 원소 (data element)
 - 부모 노드에 대한 연결 (link to parent)
 - 자식 노드들의 집합 (set of children)



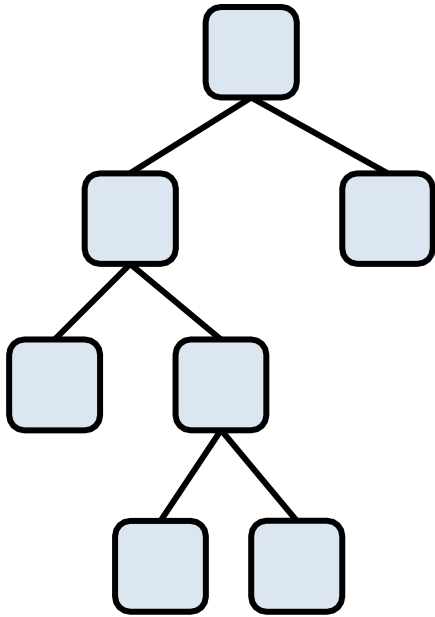
Binary Tree (이진 트리)

- 각 내부 노드는 최대 두 개의 자식을 가짐
- 한 노드의 자식들은 순서가 정해져 있음

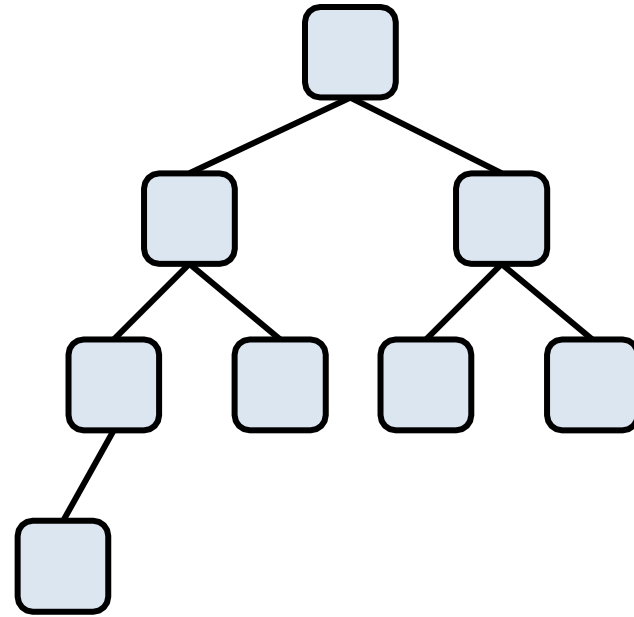


Types of Binary Trees

- 정 이진 트리 (Full Binary Tree)
 - 각 노드의 자식 노드 수가 2 또는 0인 트리
- 완전 이진 트리 (Complete Binary Tree)
 - 가장 깊은 레벨을 제외한 모든 레벨이 가득 차 있음
 - 마지막 레벨의 노드들은 가능한 왼쪽에 존재



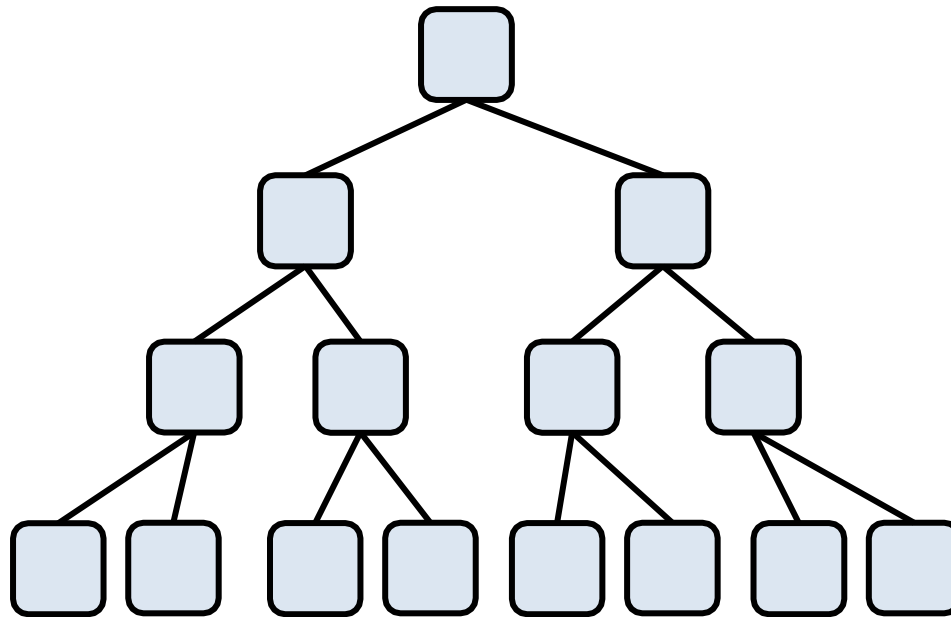
Full Binary Tree



Complete Binary Tree

Types of Binary Trees

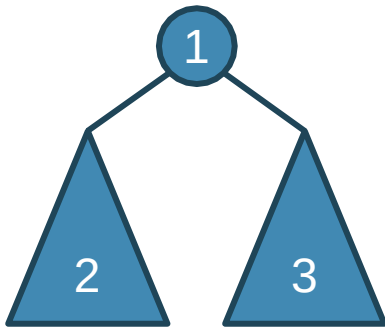
- 포화 이진 트리 (Perfect Binary Tree)
 - 모든 리프 노드의 레벨이 서로 같음
 - 모든 내부 노드의 자식 노드 수가 2임



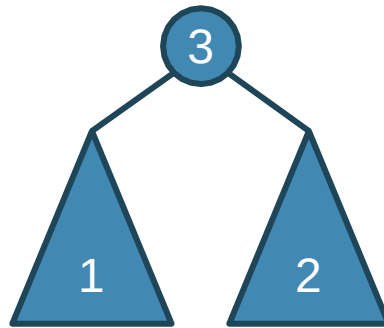
Perfect Binary Tree

Tree Traversal (트리 순회)

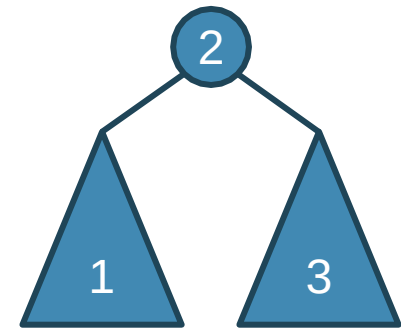
- 트리 순회
 - 트리의 모든 노드를 일정한 규칙에 따라 정확히 한번씩 방문하는 과정
- 순회 종류
 - 전위 순회 (Pre-order): 노드를 방문하고, 자식들을 순회
 - 후위 순회 (Post-order): 자식들을 방문하고, 노드를 방문
 - 중위 순회 (In-order): 왼쪽 서브트리 방문 후, 노드 방문, 그리고 나서 오른쪽 서브트리 방문



Pre-order traversal



Post-order traversal



In-order traversal

Pre-order vs. Post-order

Pre-order traversal

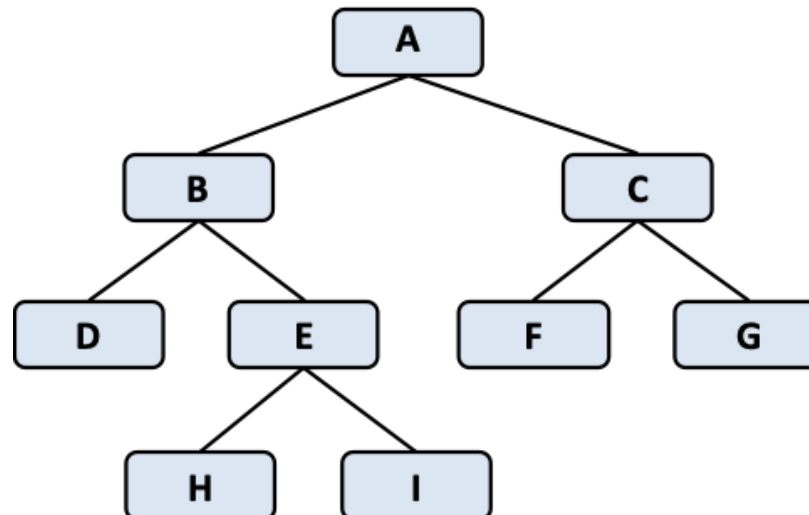
Algorithm binaryPreorder(v)

```
1: visit(v)
2: if hasLeft(v)
3:   binaryPreorder(left(v))
4: if hasRight(v)
5:   binaryPreorder(right(v))
```

Post-order traversal

Algorithm binaryPostorder(v)

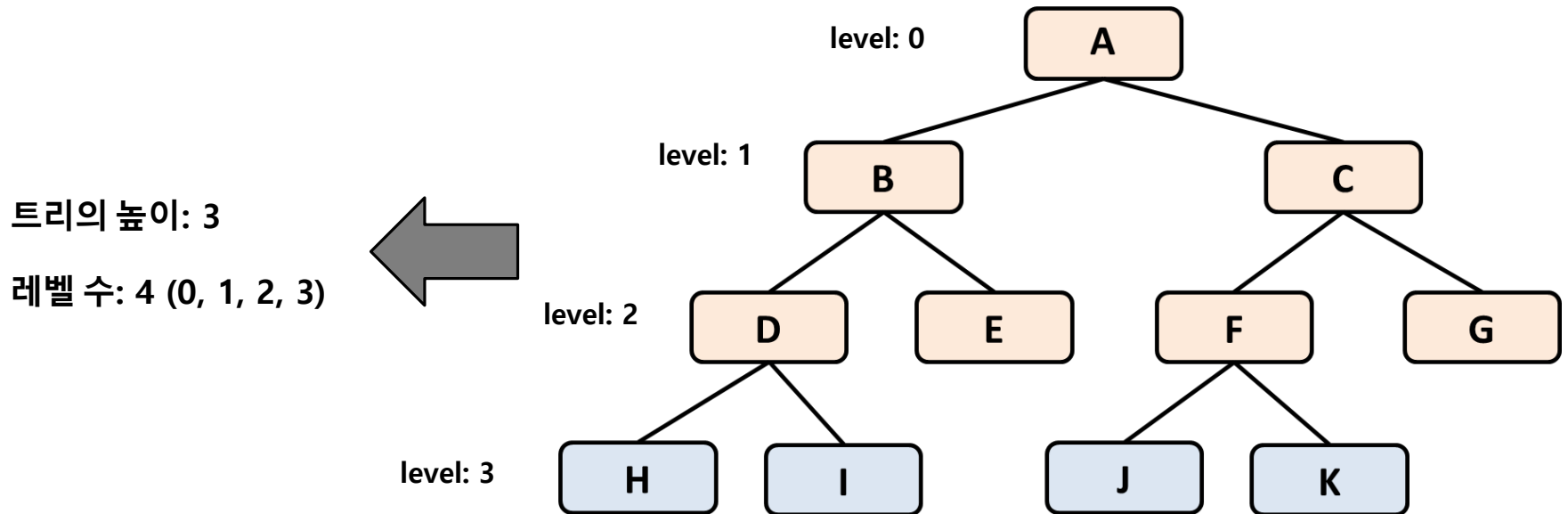
```
1: if hasLeft(v)
2:   binaryPostorder(left(v))
3: if hasRight(v)
4:   binaryPostorder(right(v))
5: visit(v)
```



Binary Tree

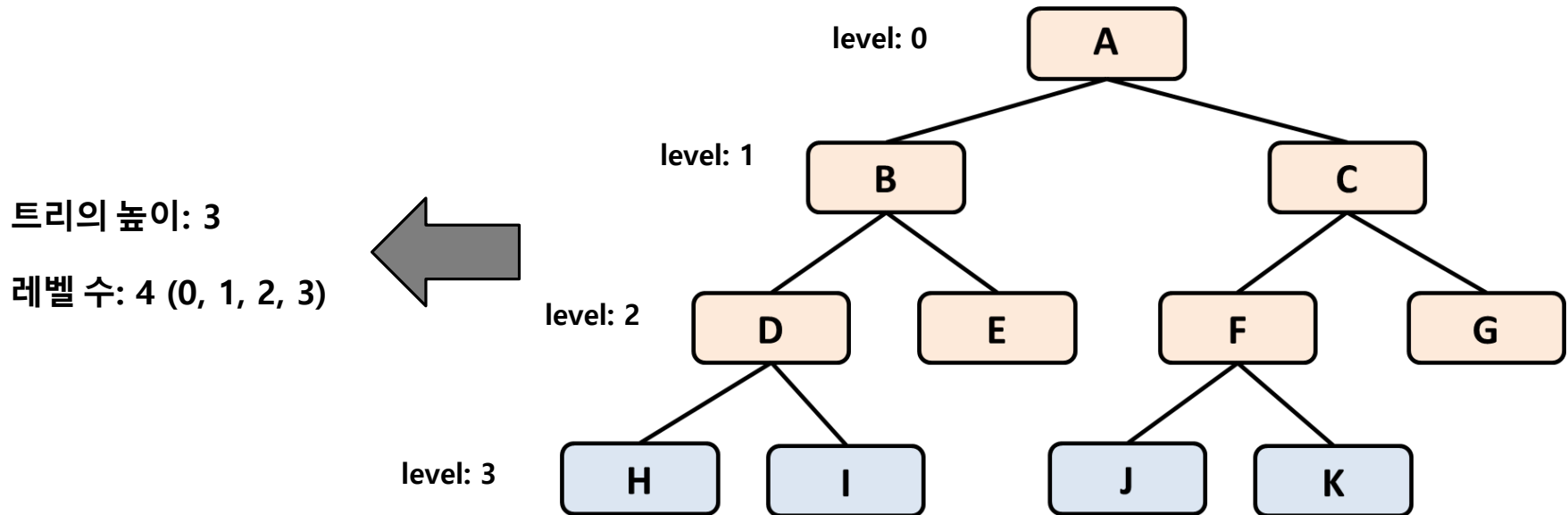
Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i



Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$



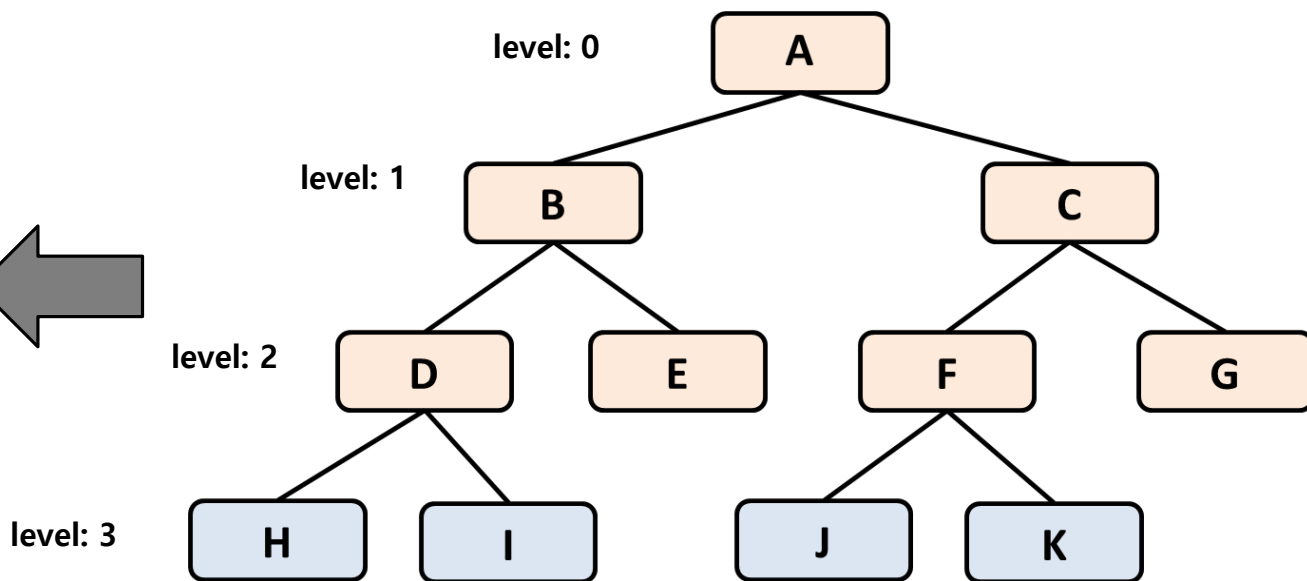
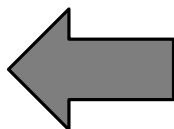
Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$
- 노드가 n 개 있을 때, 트리의 최소 레벨 수(= 높이+1): $\lceil \log_2(n + 1) \rceil$

※ $\lceil x \rceil$: x 와 같거나 x 보다 큰 정수 중 가장 작은 값 (ceiling function)

트리의 높이: 3

레벨 수: 4 (0, 1, 2, 3)

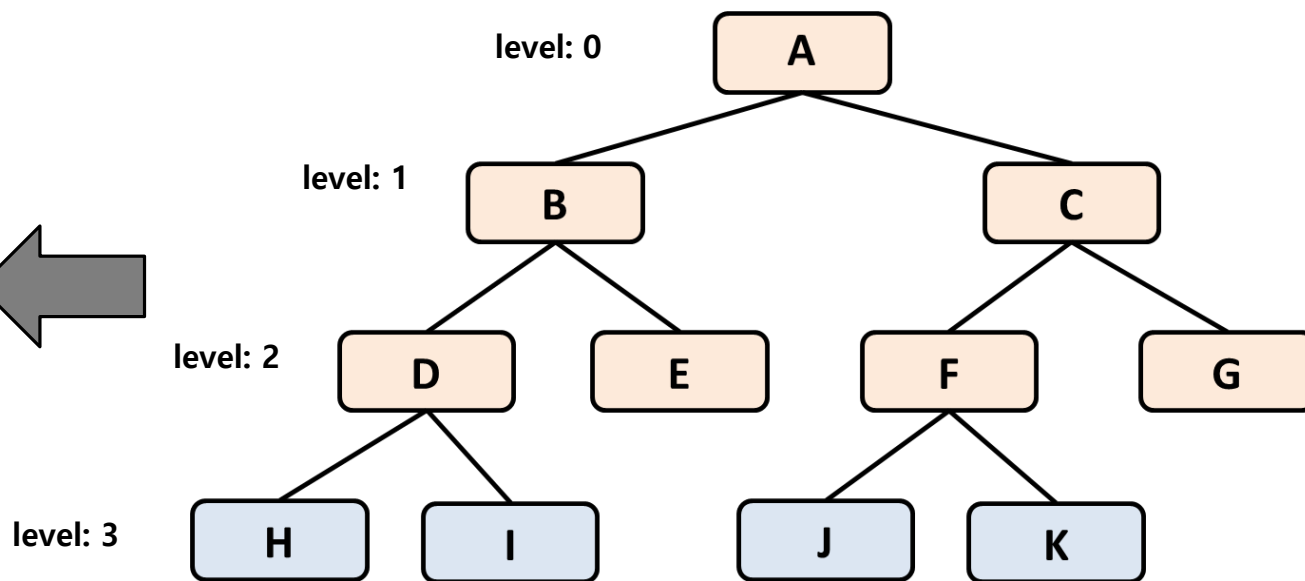
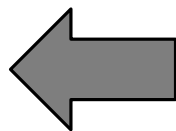


Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i ※ $\lceil x \rceil$: x 와 같거나 x 보다 큰 정수 중 가장 작은 값 (ceiling function)
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$
- 노드가 n 개 있을 때, 트리의 최소 레벨 수(= 높이+1): $\lceil \log_2(n + 1) \rceil$
 - 높이가 h 인 이진 트리의 최대 노드 수: $2^{h+1} - 1$

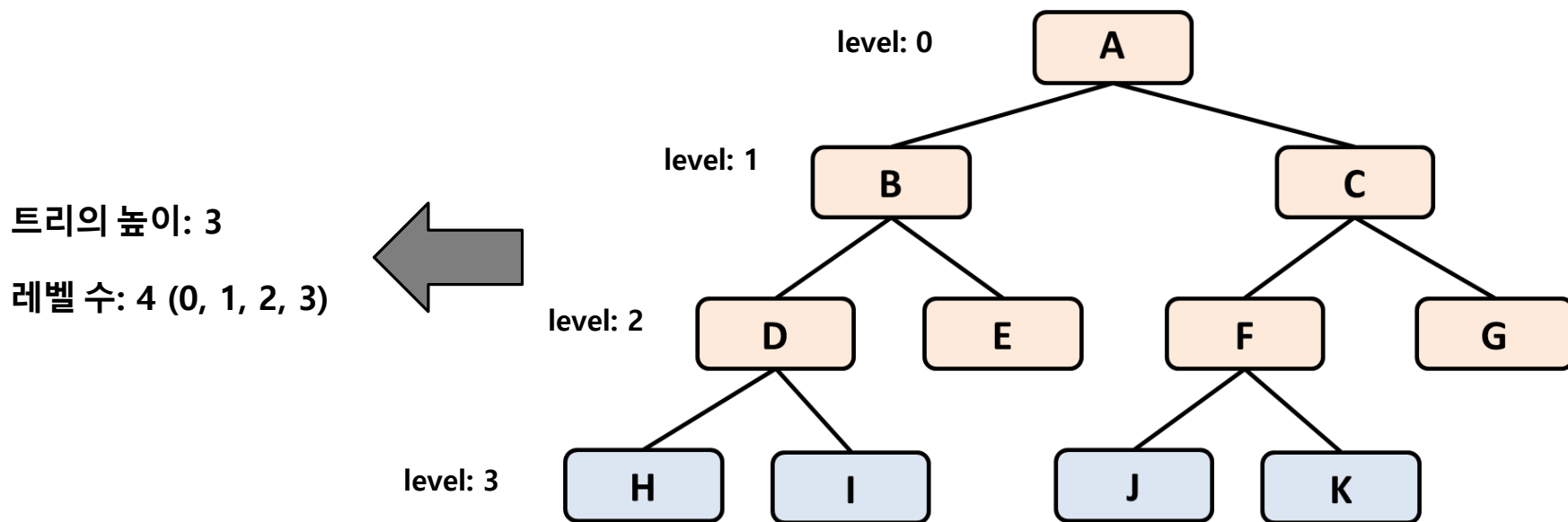
트리의 높이: 3

레벨 수: 4 (0, 1, 2, 3)



Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$
- 노드가 n 개 있을 때, 트리의 최소 레벨 수(= 높이+1): $\lceil \log_2(n + 1) \rceil$
 - 높이가 h 인 이진 트리의 최대 노드 수: $2^{h+1} - 1$
 $2^{h+1} - 1 \geq n \rightarrow h + 1 \geq \log_2(n + 1)$

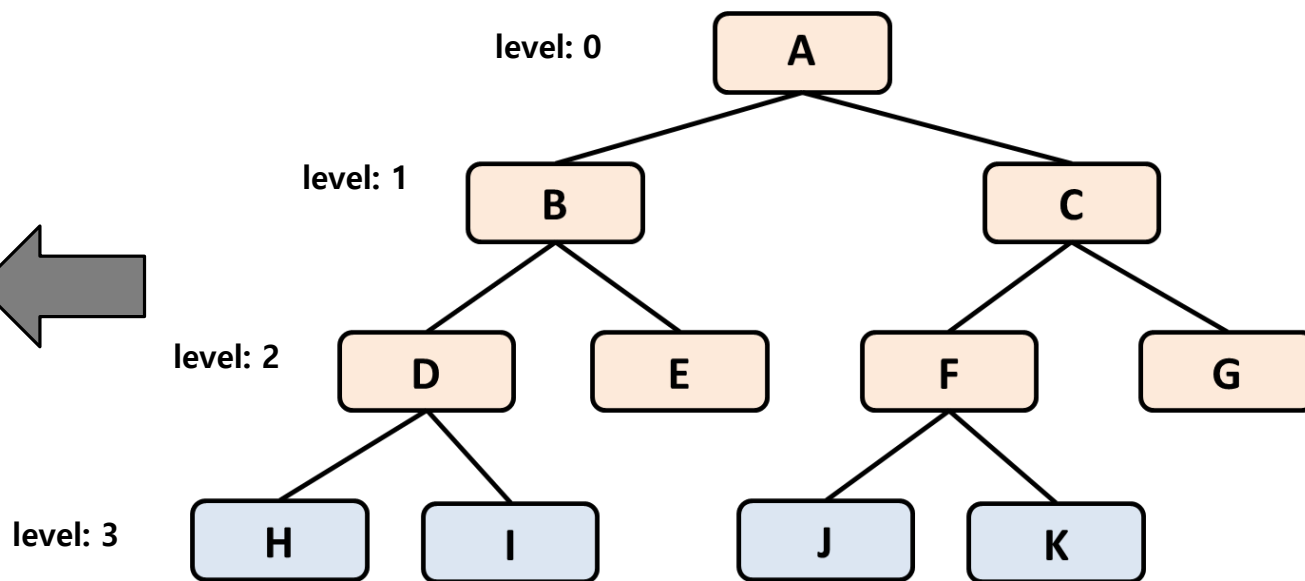
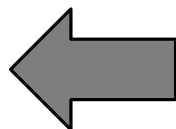


Binary Tree: 노드 수 관련 속성

- 레벨 i 에서의 최대 노드 수: 2^i ※ $\lceil x \rceil$: x 와 같거나 x 보다 큰 정수 중 가장 작은 값 (ceiling function)
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$
- 노드가 n 개 있을 때, 트리의 최소 레벨 수(= 높이+1): $\lceil \log_2(n + 1) \rceil$
 - 높이가 h 인 이진 트리의 최대 노드 수: $2^{h+1} - 1$
 $2^{h+1} - 1 \geq n \rightarrow h + 1 \geq \log_2(n + 1)$
 - 레벨 수는 (높이+1)과 같으므로, 최소 레벨 수는 $\lceil \log_2(n + 1) \rceil$.

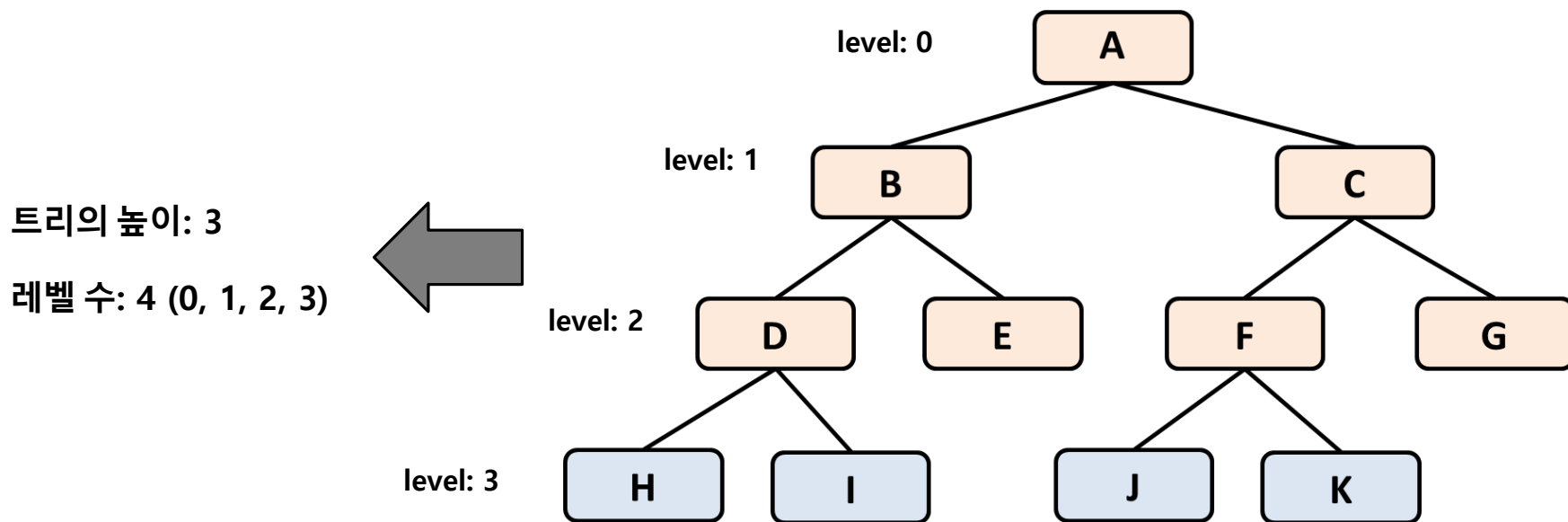
트리의 높이: 3

레벨 수: 4 (0, 1, 2, 3)

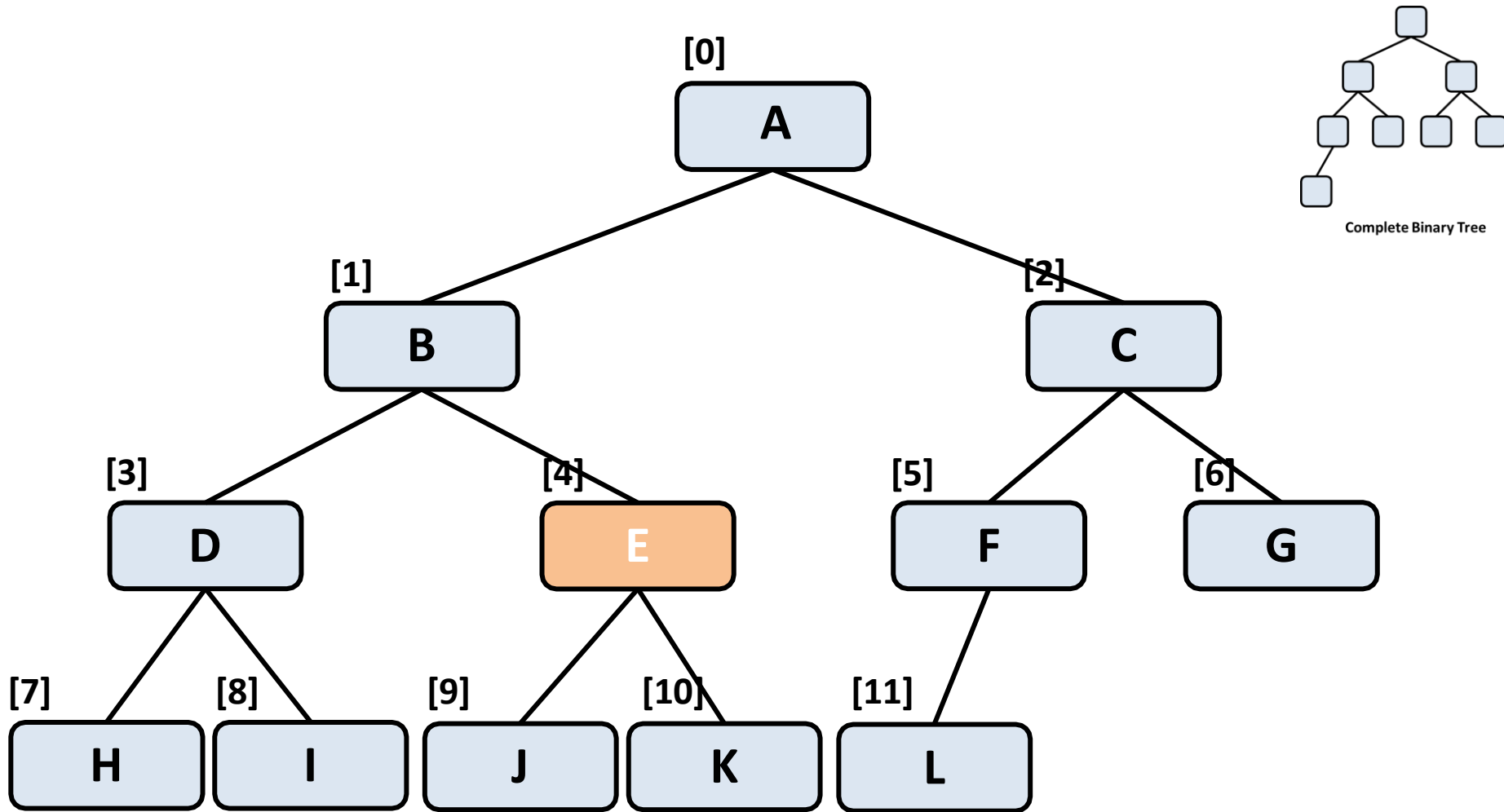


Binary Tree: 노드 수 관련 속성

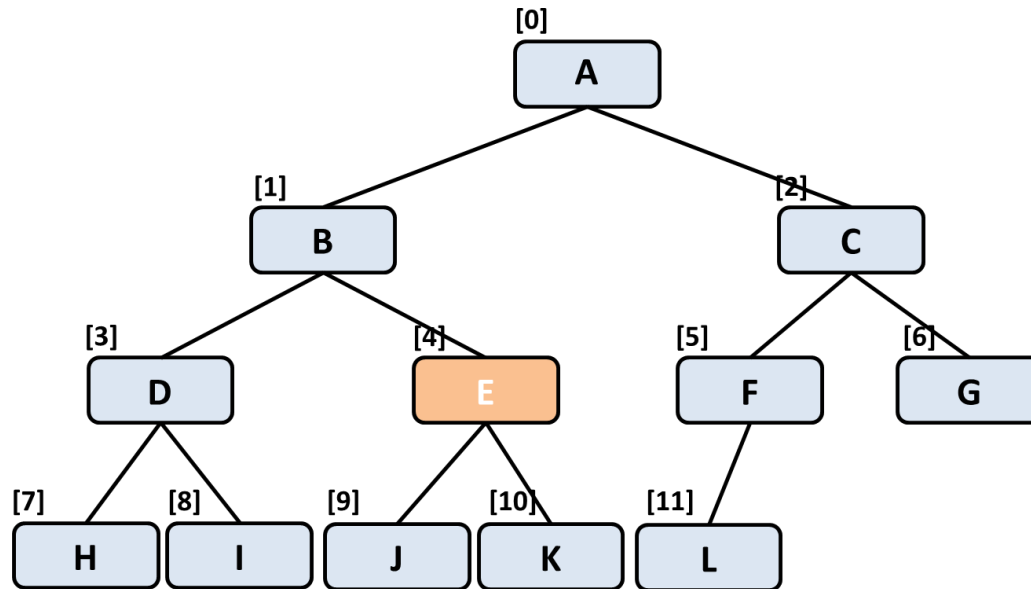
- 레벨 i 에서의 최대 노드 수: 2^i
- 트리 높이가 h 일 때, 최대 노드 수: $1 + 2^1 + \dots + 2^h = 2^{h+1} - 1$
- 노드가 n 개 있을 때, 트리의 최소 레벨 수(= 높이+1): $\lceil \log_2(n + 1) \rceil$
 - 높이가 h 인 이진 트리의 최대 노드 수: $2^{h+1} - 1$
 $2^{h+1} - 1 \geq n \rightarrow h + 1 \geq \log_2(n + 1)$
 - 레벨 수는 (높이+1)과 같으므로, 최소 레벨 수는 $\lceil \log_2(n + 1) \rceil$.
- 노드가 n 개 있을 때, 간선(edge)의 수: $n - 1$



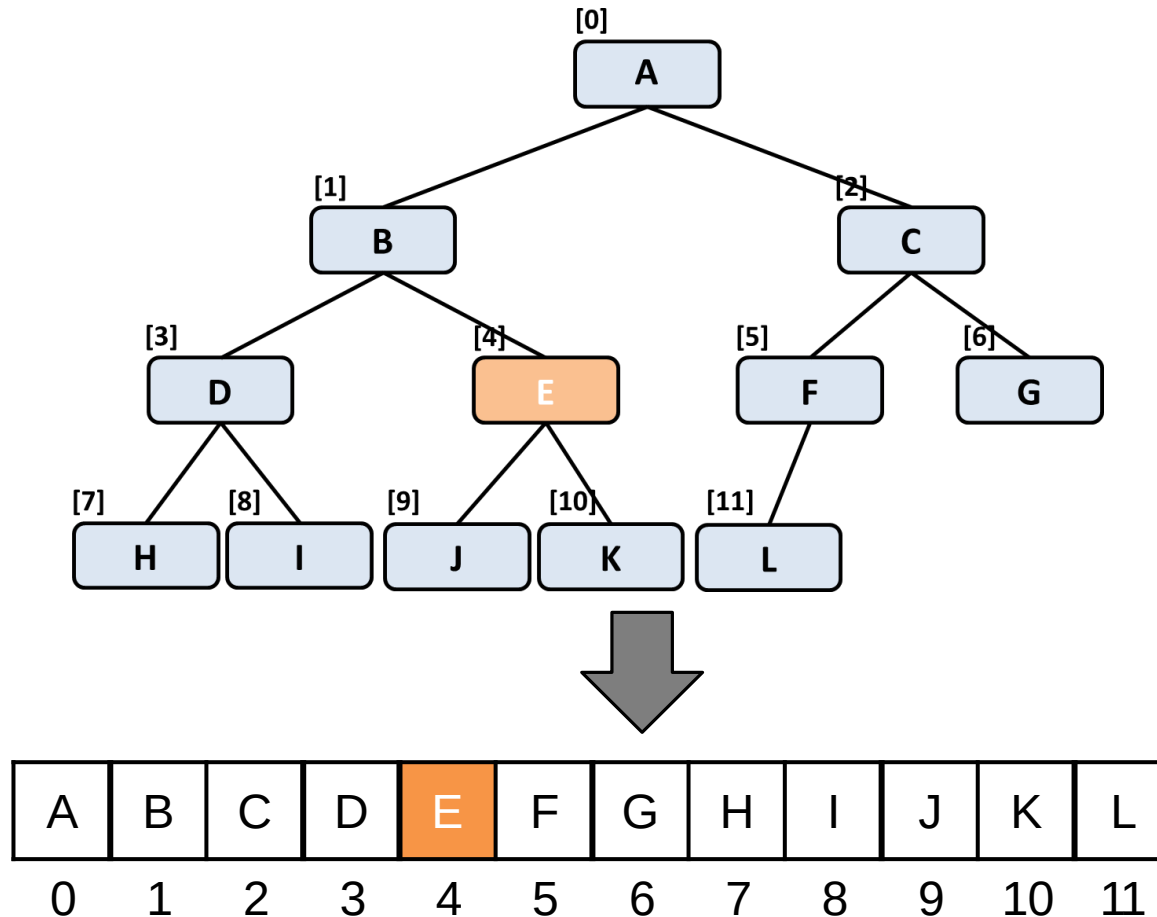
배열 기반 구현: 완전(Complete) 이진 트리



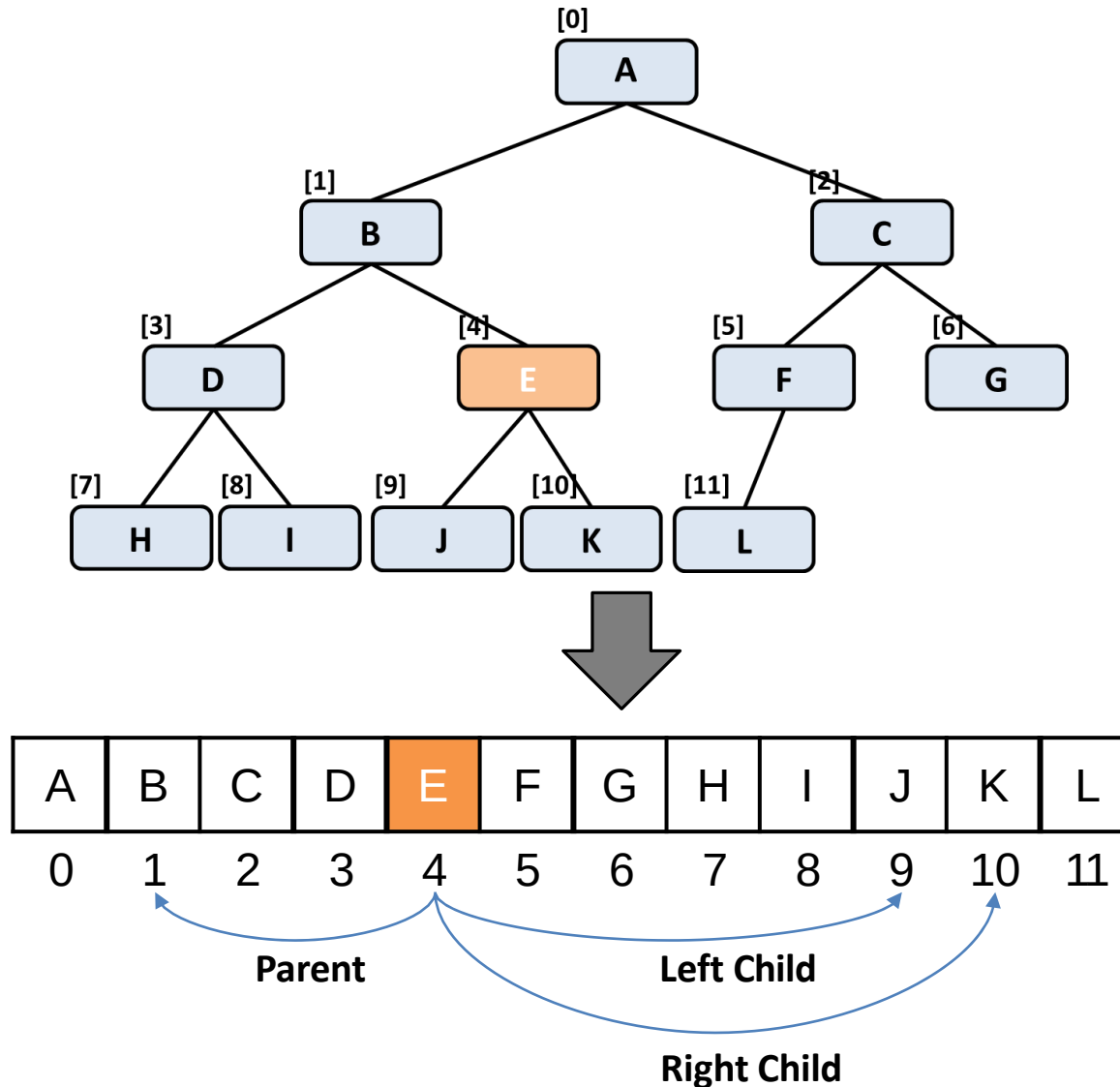
배열 기반 구현: 완전(Complete) 이진 트리



배열 기반 구현: 완전(Complete) 이진 트리



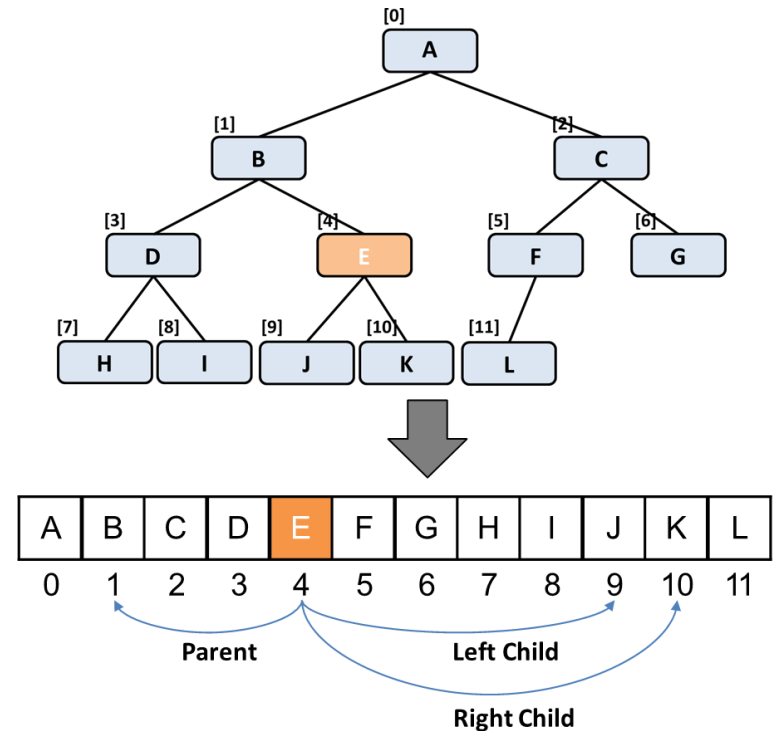
배열 기반 구현: 완전(Complete) 이진 트리



배열 기반 구현: 완전(Complete) 이진 트리

- $parent(i) = \lfloor (i - 1)/2 \rfloor$ (if $i > 0$)

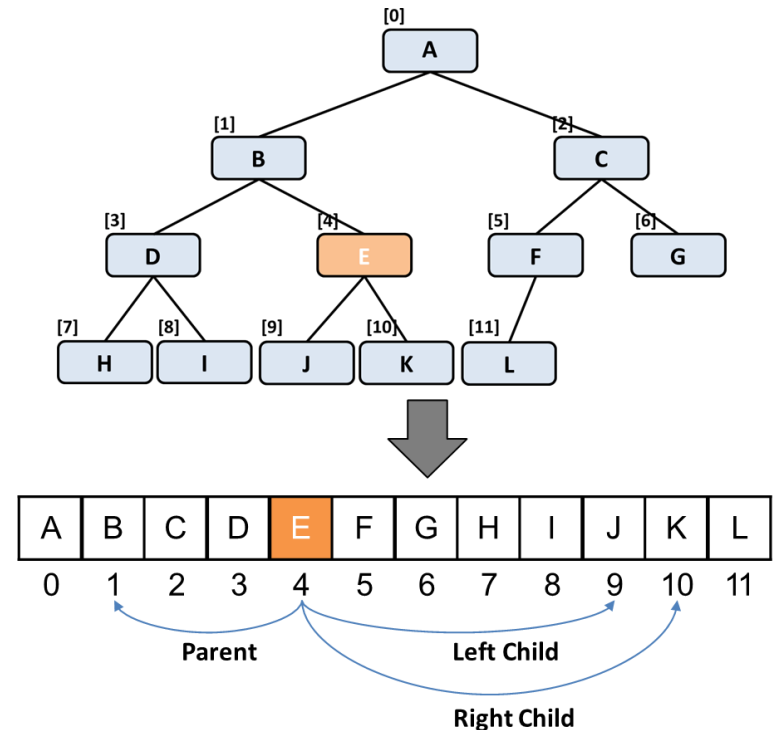
※ $\lfloor x \rfloor$: x 와 같거나 x 보다 작은 정수 중 가장 큰 값 (floor function)



배열 기반 구현: 완전(Complete) 이진 트리

- $parent(i) = \lfloor (i - 1) / 2 \rfloor$ (if $i > 0$)
- $leftChild(i) = 2i + 1$ (if $2i + 1 < n$)
- $rightChild(i) = 2i + 2$ (if $2i + 2 < n$)

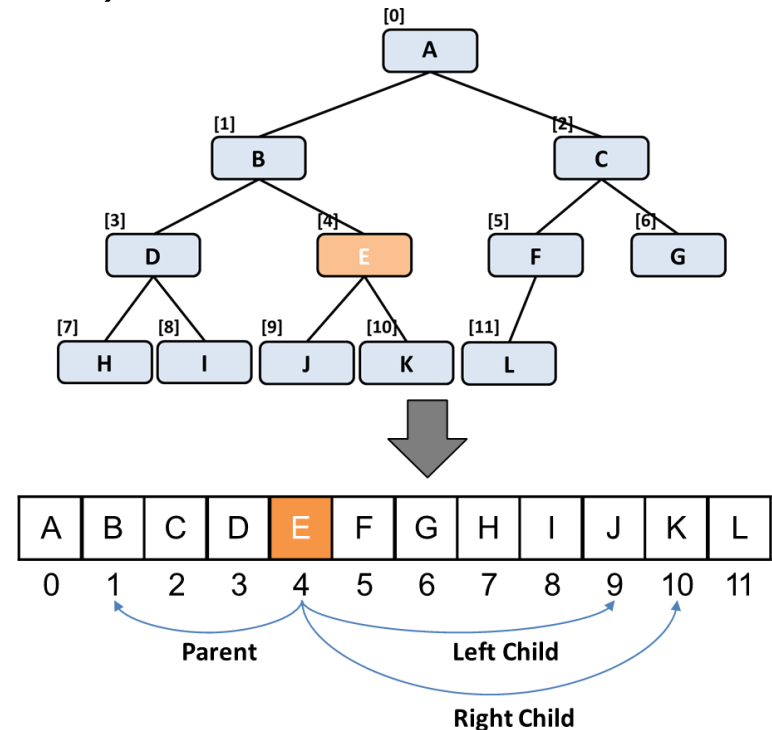
※ $\lfloor x \rfloor$: x 와 같거나 x 보다 작은 정수 중 가장 큰 값 (floor function)



배열 기반 구현: 완전(Complete) 이진 트리

- $parent(i) = \lfloor (i - 1) / 2 \rfloor$ (if $i > 0$)
- $leftChild(i) = 2i + 1$ (if $2i + 1 < n$)
- $rightChild(i) = 2i + 2$ (if $2i + 2 < n$)
- $leftSibling(i) = i - 1$ (if i is even)
- $rightSibling(i) = i + 1$ (if i is odd & $i + 1 < n$)

※ $\lfloor x \rfloor$: x 와 같거나 x 보다 작은 정수 중 가장 큰 값 (floor function)



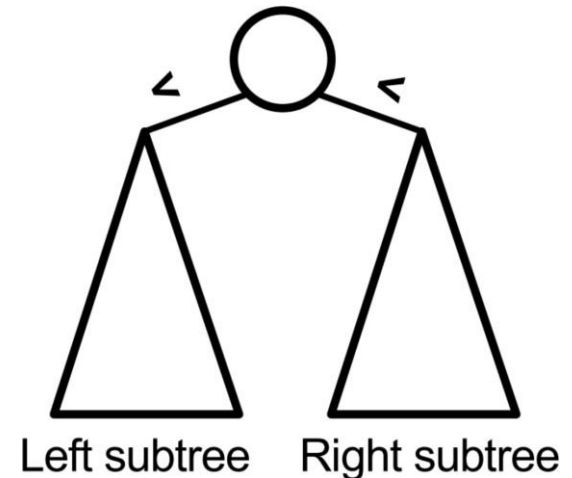
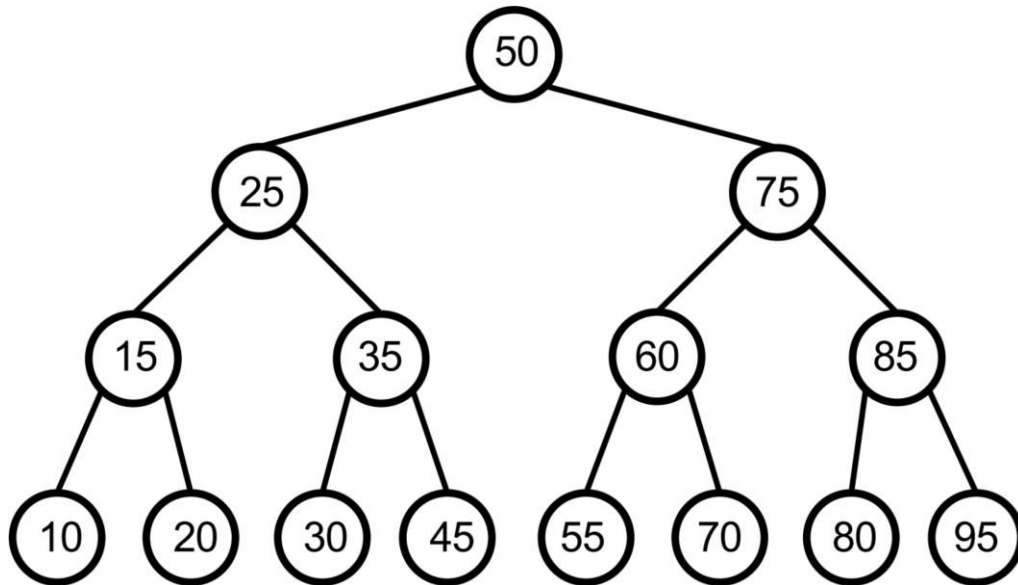
Binary Search Trees

Definition: Binary Search Trees

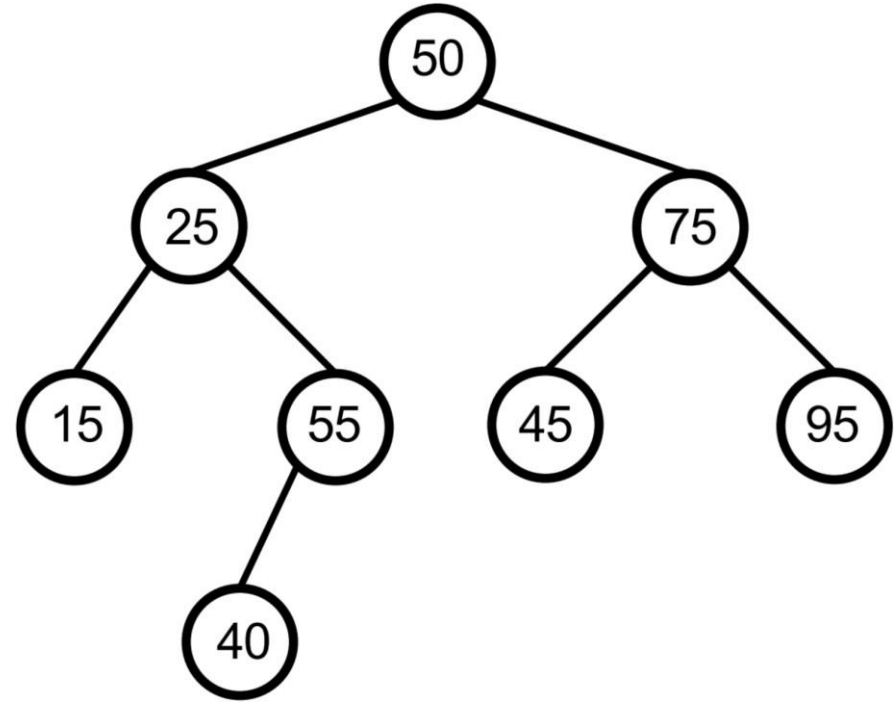
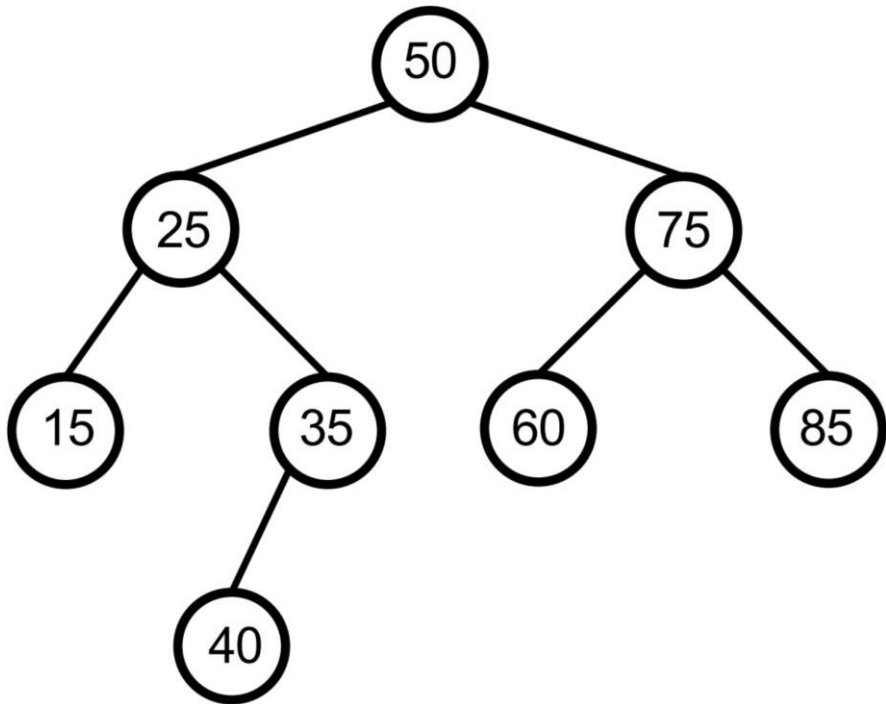
- 이진 탐색 트리(BST): 다음 조건을 따르는 이진 트리
 - 모든 노드는 **유일한 키**를 가짐
 - K라는 키를 갖는 노드의 **왼쪽 subtree**에 있는 모든 노드는 **K보다 작은 키**를 가짐
 - K라는 키를 갖는 노드의 **오른쪽 subtree**에 있는 모든 노드는 **K보다 큰 키**를 가짐
 - 모든 subtree는 BST임

Definition: Binary Search Trees

- 이진 탐색 트리(BST): 다음 조건을 따르는 이진 트리
 - 모든 노드는 **유일한 키**를 가짐
 - K라는 키를 갖는 노드의 **왼쪽 subtree**에 있는 모든 노드는 **K보다 작은 키**를 가짐
 - K라는 키를 갖는 노드의 **오른쪽 subtree**에 있는 모든 노드는 **K보다 큰 키**를 가짐
 - 모든 subtree는 BST임

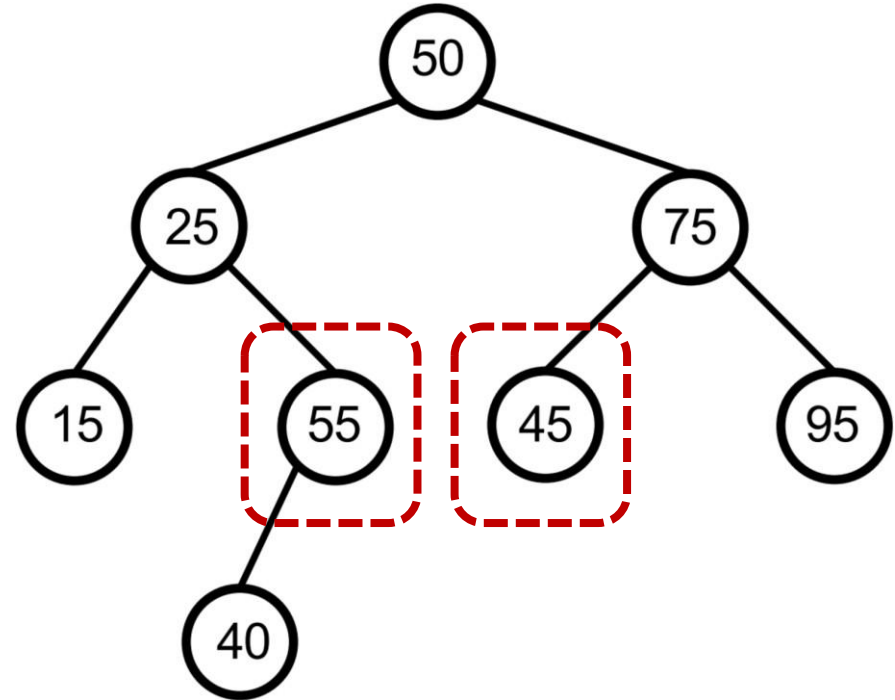
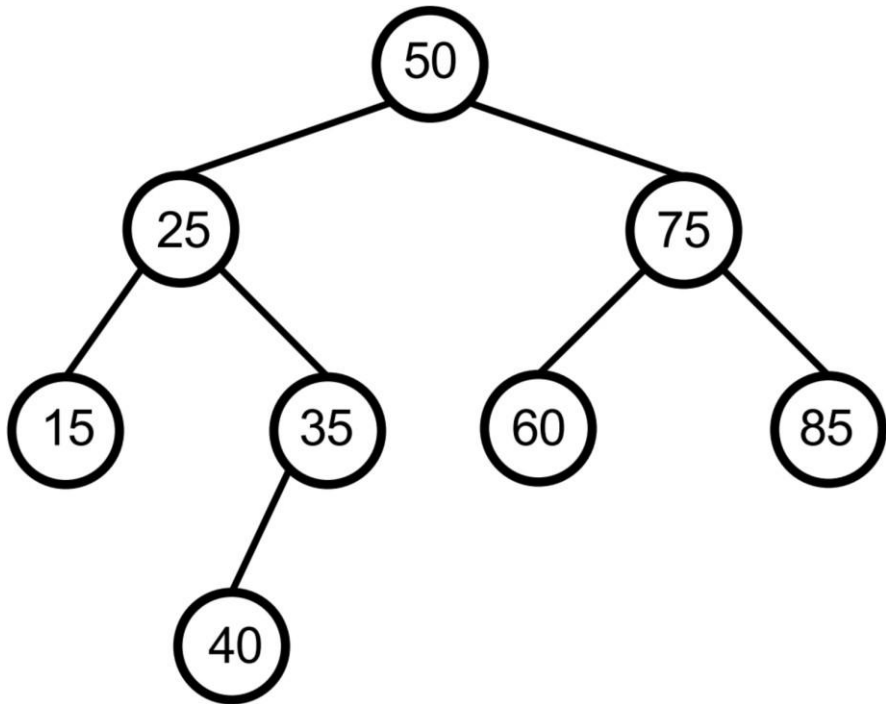


Binary Search Trees?



“두 트리는 이진 탐색 트리가 되기 위한 조건을 만족하는가?”

Binary Search Trees?



“두 트리는 이진 탐색 트리가 되기 위한 조건을 만족하는가?”

ADT: Binary Search Tree

```
public interface BST<Key extends Comparable<Key>, E> {  
    /** Reinitialize BST: remove all records */  
    public void clear();  
  
    /** Insert a record with key k and element e */  
    public void insert(Key k, E e);  
  
    /** Remove and return the record matching key k (null if not found) */  
    public E remove(Key k);  
  
    /** Remove and return the record with the smallest key (null if tree is empty) */  
    public E removeMin();  
  
    /** Remove and return the record with the largest key (null if tree is empty) */  
    public E removeMax();  
  
    /** Return a record matching key k (null if not found) */  
    public E find(Key k);  
  
    /** @return The number of records currently stored in the BST */  
    public int size();  
};
```

ADT: Binary Search Tree

값에 대한 비교가 가능한 타입만이 key로써 사용되도록 제약

```
public interface BST<Key extends Comparable<Key>, E> {  
    /** Reinitialize BST: remove all records */  
    public void clear();  
  
    /** Insert a record with key k and element e */  
    public void insert(Key k, E e);  
  
    /** Remove and return the record matching key k (null if not found) */  
    public E remove(Key k);  
  
    /** Remove and return the record with the smallest key (null if tree is empty) */  
    public E removeMin();  
  
    /** Remove and return the record with the largest key (null if tree is empty) */  
    public E removeMax();  
  
    /** Return a record matching key k (null if not found) */  
    public E find(Key k);  
  
    /** @return The number of records currently stored in the BST */  
    public int size();  
};
```

Operations: Search

- Key가 x 인 노드 찾기

Operations: Search

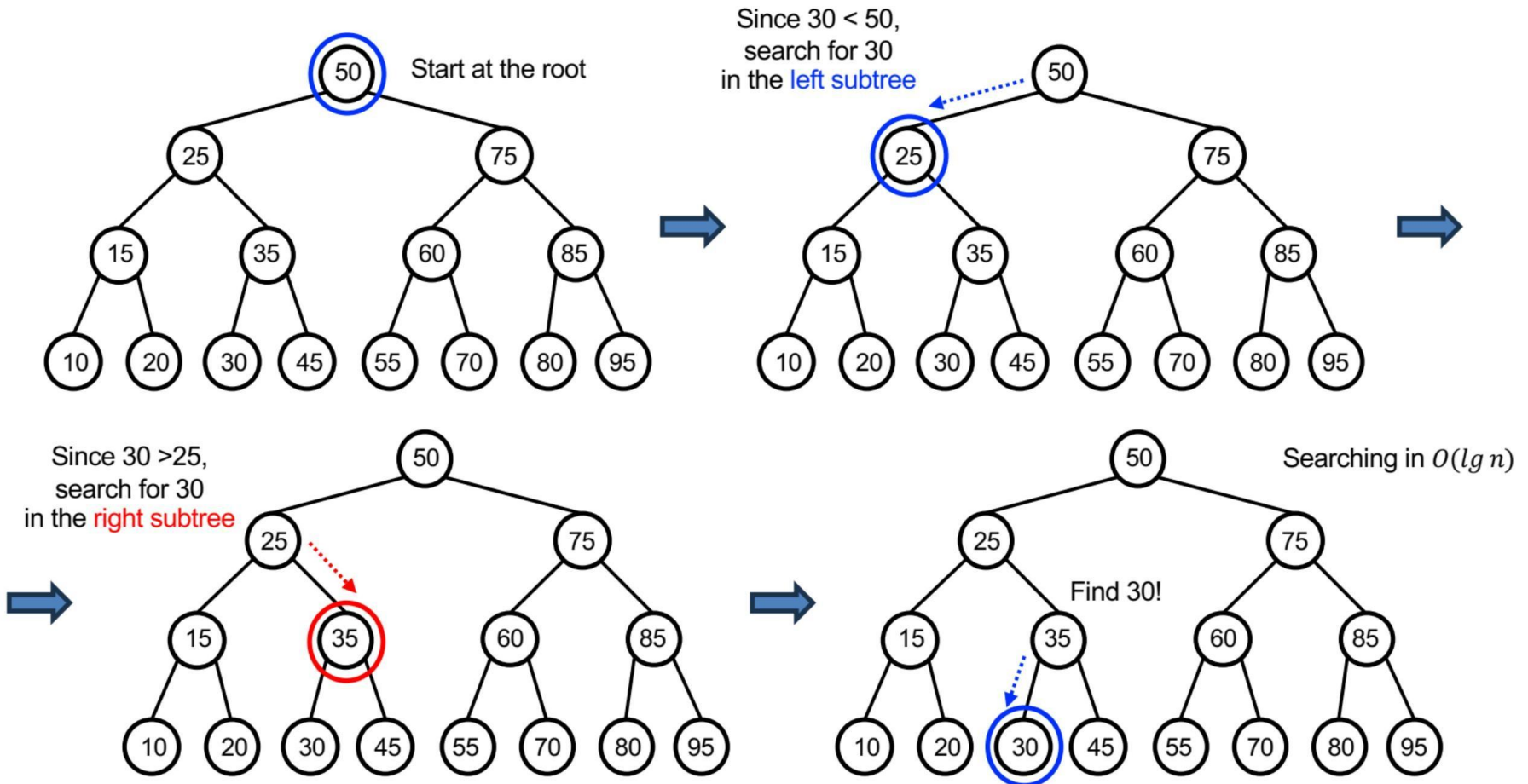
- Key가 x 인 노드 찾기
 - (1) 루트 노드에서 시작
 - (2) x 와 루트 노드의 key ($rt.key$) 간의 비교를 수행:
 - If $x = rt.key$, 노드 rt 를 리턴

Operations: Search

- Key가 x 인 노드 찾기
 - (1) 루트 노드에서 시작
 - (2) x 와 루트 노드의 key ($rt.key$) 간의 비교를 수행:
 - If $x = rt.key$, 노드 rt 를 리턴
 - If $x < rt.key$, 노드 rt 의 왼쪽 서브트리에서 다시 찾기
 - If $x > rt.key$, 노드 rt 의 오른쪽 서브트리에서 다시 찾기

Operations: Search

- Ex) Key가 30인 노드 찾기



Operations: Search (코드)

```
/** Return a record matching key k (null if not found) */
public E find(Key k) {      // 외부에서 호출하는 메서드
    return findHelper(root, k);    // 트리 최상단 노드(root)부터 탐색 시작
}

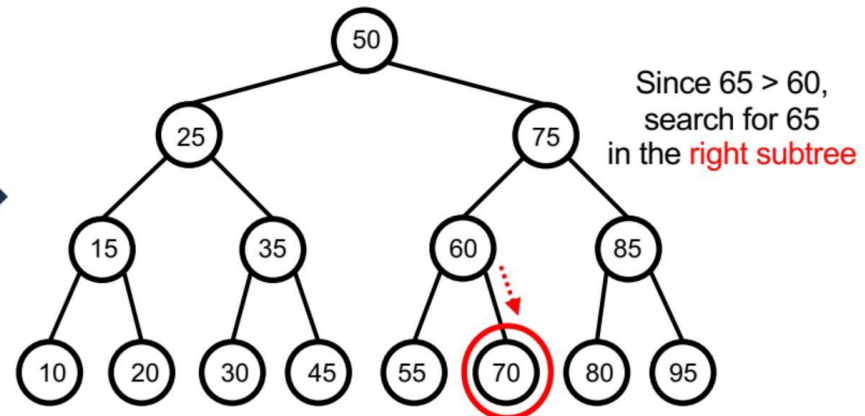
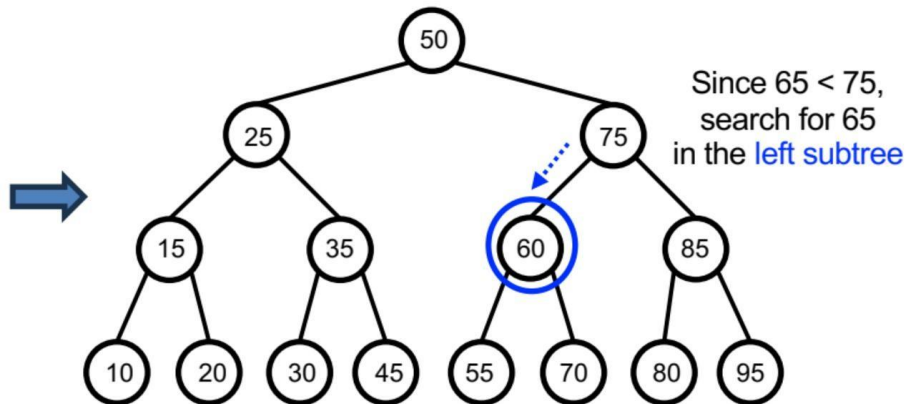
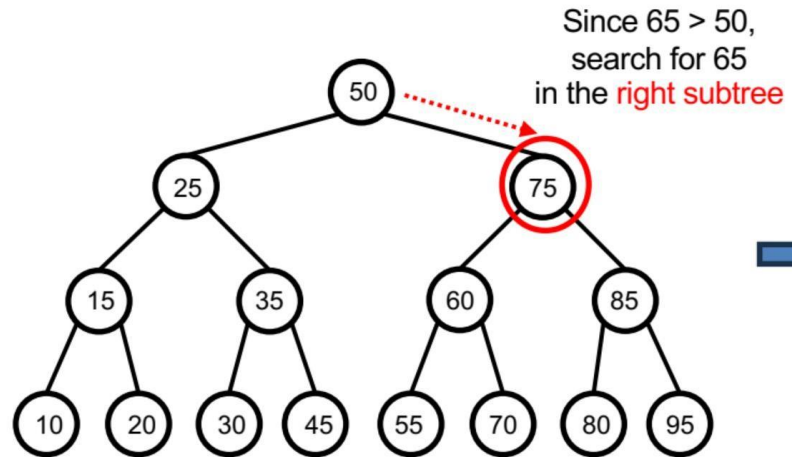
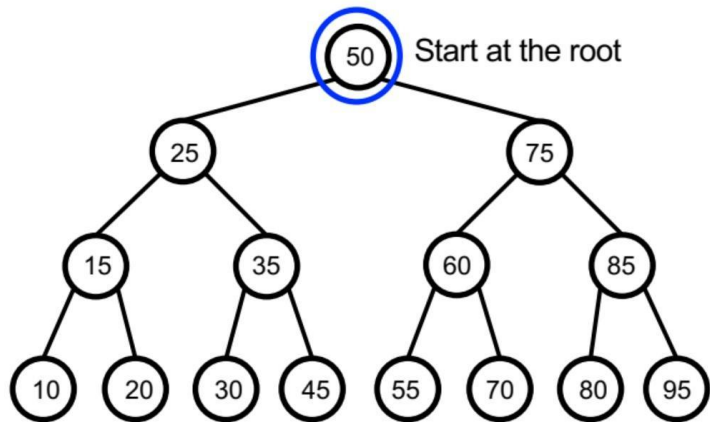
// 실제 탐색을 수행하는 재귀 메서드
private E findHelper(Node rt, Key k) {
    if (rt == null) return null;
    if (rt.key.compareTo(k) == 0)    // rt.key == k
        return rt.element;
    else if (rt.key.compareTo(k) > 0)    // rt.key > k
        return findHelper(rt.left, k);
    else    // rt.key < k
        return findHelper(rt.right, k);
}
```

Operations: Insert

- Key가 x 인 노드 넣기
 - (1) Search 연산 수행
 - 만일 key가 x 인 노드가 이미 있을 경우, element 업데이트
 - (2) Search가 실패한 위치에 key가 x 인 노드 삽입

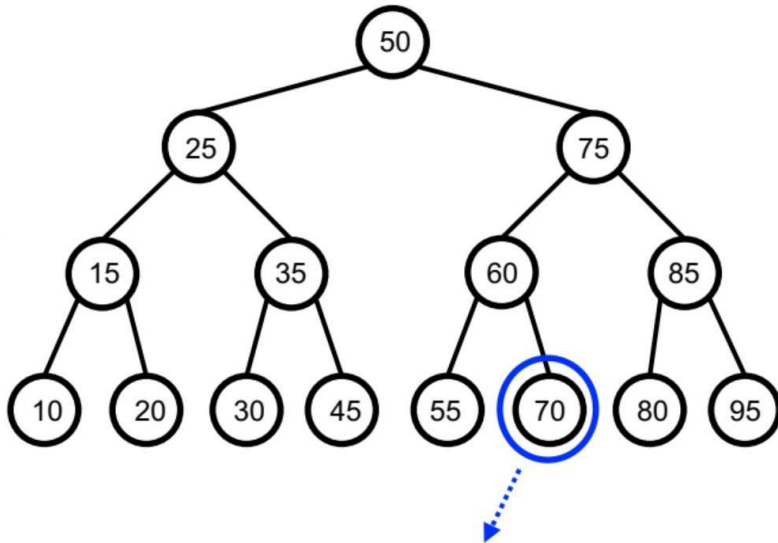
Operations: Insert

- Ex) Key가 65인 노드 넣기

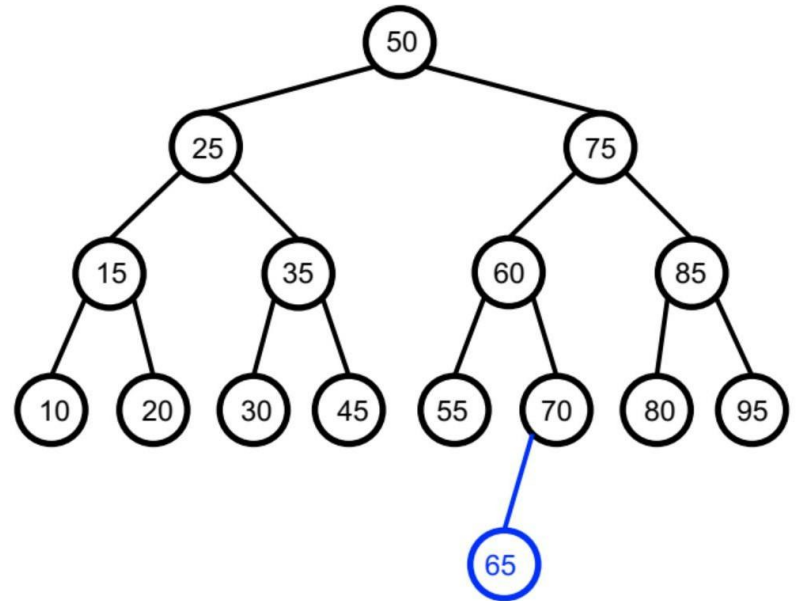


Operations: Insert

- Ex) Key가 65인 노드 넣기



Since $65 < 70$,
search for 65
in the **left subtree**



Since '70' has no child,
Insert '65' as the **left child** of the node 70

Operations: Insert (코드)

```
/** Insert a record with key k and element e */
public void insert(Key k, E e) {
    root = insertHelper(root, k, e);
    nodeCount++;
}

private Node insertHelper(Node rt, Key k, E e) {
    if (rt == null) return new Node(k, e); // 빈 자리 찾음; 새 노드 삽입

    int cmp = rt.key.compareTo(k);
    if (cmp > 0) // rt.key > k
        rt.left = insertHelper(rt.left, k, e);
    else if (cmp < 0) // rt.key <= k
        rt.right = insertHelper(rt.right, k, e);
    else // rt.key == k
        rt.element = e; // 기존 값 덮어쓰임
    return rt;
}
```

Operations: Insert (코드)

```
/** Insert a record with key k and element e */
public void insert(Key k, E e) {
    root = insertHelper(root, k, e);
    nodeCount++;
}

private Node insertHelper(Node rt, Key k, E e) {
    if (rt == null) return new Node(k, e); // 빈 자리 찾음; 새 노드 삽입

    int cmp = rt.key.compareTo(k);
    if (cmp > 0) // rt.key > k
        rt.left = insertHelper(rt.left, k, e);
    else if (cmp < 0) // rt.key <= k
        rt.right = insertHelper(rt.right, k, e);
    else // rt.key == k
        rt.element = e; // 기존 값 덮어쓰임
    return rt;
}
```

최종적으로 항상 전체 트리의 루트 노드가 반환됨

(새로운 노드 삽입 후, 재귀호출이 끝나면서 각 단계에서의 서브트리의 루트가 반환.
이 과정이 재귀적으로 반복됨으로써, 최종적으로 전체 트리의 루트가 반환됨)

Look ahead

Binary Search Tree, AVL Tree